Google Developer Groups

On Campus • Fr. Conceicao Rodrigues College of Engineering

AROUND THE WORLD

BIT N BUILD



COLLEGE ROUND

Google Developer Groups
On Campus • Fr. Conceicao Rodrigues College of Engineering

AROUND THE WORLD

BIT N BUILD



Web /App Dev

1. Nexus: The AI-Powered Business Operating Platform

Backstory:

Businesses often rely on separate tools for website creation, customer management, marketing, subscriptions, and online events. These disconnected systems create fragmented data, repetitive manual work, and inefficient workflows. Although AI tools can assist with individual tasks, they rarely operate across an entire business workflow.

Problem Statement:

Develop Nexus, an AI-powered business management platform that unifies website creation, CRM, marketing, subscription management, and webinar operations within a single workspace.

The platform must allow businesses to build websites using a visual editor, manage leads and customers, schedule marketing campaigns, host webinars, and handle recurring or usage-based subscriptions. An AI agent must execute authorized, multi-step business workflows through natural-language instructions, coordinating actions across modules while maintaining data consistency and requiring approval for sensitive operations.

Teams must demonstrate an integrated workflow that goes beyond isolated features and show how AI-driven automation reduces manual effort across the business lifecycle.

Key Features:

- Visual Website Builder: Create and customize responsive websites without coding.
- Intelligent CRM: Manage leads, customer records, registrations, and conversion tracking.
- SaaS Billing: Support subscription plans, recurring payments, and usage-based pricing.
- Marketing Automation: Plan content, schedule campaigns, and track engagement.
- Webinar Management: Handle event creation, registrations, attendance, and follow-ups.
- AI Business Agent: Execute multi-step workflows across authorized platform modules using natural-language instructions.
- Workflow Security: Enforce permissions, approval checkpoints, and reliable execution of business actions.
- Integrated Analytics: Track leads, conversions, subscriptions, campaign performance, and workflow outcomes.

2. Handsfree: Voice and Webcam-Based Browsing

Backstory:

Millions of people struggle to use a mouse or keyboard comfortably due to motor impairments, temporary injuries, or repetitive strain. While assistive devices exist, they can be expensive or require specialized hardware. Webcam tracking and voice recognition offer accessible alternatives, but making them accurate, reliable, and easy to use together remains challenging.

Problem Statement:

Develop Handsfree, a browsing solution that lets users interact with websites using a standard webcam and microphone, without specialized hardware. Users should be able to control the cursor through head or hand movements, navigate using voice commands, and dictate text into input fields. The system must minimize accidental actions and adapt to changing lighting, posture, and background noise.

Key Features:

- Cursor Control: Translate head or hand movements into smooth, responsive cursor movement.
- Voice Navigation: Support commands for clicking, scrolling, opening links, and navigating between pages or tabs.
- Voice Typing: Dictate text into search bars, forms, and other input fields.
- Intentional Activation: Prevent accidental clicks using dwell selection, gestures, or voice confirmation.
- Combined Interaction: Allow users to point at an element and issue a voice command to interact with it.
- Quick Calibration: Enable setup within 30 seconds and maintain control as the user changes position.
- Website Compatibility: Support browsing, navigation, and text entry through a browser extension or dedicated browsing environment.
- Performance Evaluation: Measure task completion time, command accuracy, selection errors, and accidental activations against conventional mouse and keyboard input.

3. PATCHWORK - Hotfix the Internet

Backstory:

A critical button breaks after a deployment, a checkout flow stops working, or a website update invalidates an existing fix. Rebuilding and redeploying an entire application for a small change can be costly and risky, especially when the original developers are unavailable. Existing editing tools rarely provide a complete workflow for safely patching live websites, resolving conflicting changes, and recovering from failed releases.

Problem Statement:

Build PATCHWORK, a platform that enables developers to modify existing websites through independently managed patches without rebuilding the original application. The system must intelligently track changes to website elements, detect conflicting or invalid patches, verify proposed changes in isolation, and support safe deployment and rollback when something goes wrong.

Key Features:

- Visual Patching: Modify website elements, layouts, content, and supported behaviour directly through a visual interface.
- Smart Element Tracking: Identify when website updates invalidate existing patches and locate moved elements using structural and semantic context.
- Collaborative Editing: Enable multiple developers to create, review, and merge patches while detecting conflicting changes.
- Isolated Preview: Test proposed patches without affecting the published website.
- Automated Verification: Execute browser interactions and visual checks to identify regressions before deployment.
- Versioned Deployment and Rollback: Track patch releases, detect failed changes, and restore a previously working state.
- Audit Trail: Maintain a history of modifications, deployments, conflicts, and reversions.

Google Developer Groups

On Campus • Fr. Conceicao Rodrigues College of Engineering

AROUND THE WORLD

BIT N BUILD



Ai / ML

1. CrisisGrid: Can This AI Research Actually Be Reproduced?

Backstory:

AI/ML researchers publish papers claiming improvements in model accuracy, efficiency, and generalization. However, reproducing these results is often difficult due to missing configurations, incomplete documentation, inaccessible datasets, and inconsistent evaluation methods.

Researchers must manually connect papers, source code, notebooks, and experimental results to determine whether published claims hold up under independent verification. Even small changes to a model or its training configuration can produce different results, making transparent experimentation essential.

Problem Statement:

Develop CrisisGrid, an AI-powered research reproducibility platform that analyzes AI/ML papers and their supporting artifacts to reconstruct experiments, verify published claims, and identify methodological flaws.

The system should convert research methodologies into executable, traceable experiment graphs. Researchers must be able to modify individual components, create new experiment versions, and compare their results against previous versions while identifying which changes caused differences in the evaluation pipeline.

Key Features:

- Research Understanding: Analyze research papers and their supporting artifacts.
- Claim Verification: Evaluate whether published findings can be independently reproduced.
- Experiment Reconstruction: Turn research methodologies into executable experiments.
- Version Comparison: Track experiment changes and visualize their impact on results.
- Research Auditing: Identify potential flaws, inconsistencies, and reproducibility risks.
- Evidence-Based Reporting: Present findings, limitations, and experimental comparisons with supporting evidence.
- Experiment Traceability: Maintain a clear history of experiments, configurations, and results.

2. AgentBreak : The Autonomous AI Red Team

Backstory:

AI agents are increasingly being trusted with tools, private records, external services, and multi-step decision-making. However, these capabilities introduce new security risks. A malicious instruction hidden inside a document, a poorly configured tool permission, or a vulnerability in an agent's memory can cause it to perform actions beyond its intended purpose. Traditional security testing often relies on predefined attack scenarios, making it difficult to uncover complex vulnerabilities that emerge as an agent interacts with its environment.

Problem Statement:

Build AgentBreak, an autonomous AI security testing platform that actively probes AI agents to discover exploitable vulnerabilities in their instructions, memory, retrieved content, tool permissions, and multi-step execution. The platform must generate reproducible attack traces, demonstrate vulnerabilities within a controlled sandbox, and evaluate whether security defenses can prevent the same attacks without breaking legitimate agent functionality.

Key Features:

- Autonomous Attack Discovery: Identify vulnerabilities through adaptive adversarial testing rather than relying exclusively on predefined attack cases.
- Multi-Stage Attack Analysis: Trace how malicious inputs, retrieved instructions, and agent decisions can combine to produce unauthorized actions.
- Tool and Permission Auditing: Detect situations where an agent can access data or execute actions beyond its intended permissions.
- Reproducible Attack Traces: Record the inputs, intermediate decisions, tool calls, and outcomes required to reproduce a discovered vulnerability.
- Defense Validation: Evaluate proposed mitigations against previously successful attacks and measure whether they prevent exploitation.
- Security Evaluation Dashboard: Visualize attack paths, vulnerability severity, exploit success rates, and defense effectiveness.
- Regression Testing: Check whether security improvements introduce new vulnerabilities or interfere with legitimate agent behavior.

3. Second Opinion : Audit Another AI

Backstory:

AI-generated answers can sound convincing even when they contain fabricated facts, unsupported claims, or flawed reasoning. Traditional fact-checking is time-consuming, while simply asking another AI to verify an answer can reproduce the same mistakes. Reliable verification requires more than identifying suspicious statements; it requires evidence showing exactly what is wrong and why.

Problem Statement:

Build Second Opinion, an AI auditing platform that evaluates AI-generated answers against provided source material. The system must identify hallucinated facts, unsupported claims, logical inconsistencies, and potentially unsafe recommendations, pinpoint the exact portions containing errors, and justify each finding using verifiable evidence. It must distinguish genuine errors from correct or ambiguous claims while clearly communicating uncertainty.

Key Features:

- Claim-Level Verification: Break answers into individual claims and assess their consistency with the available evidence.
- Evidence-Grounded Auditing: Identify supporting, contradicting, or insufficient evidence for each claim, with references to relevant source passages.
- Precise Error Localization: Highlight the exact text containing an error and explain the nature of the problem.
- Reasoning Consistency Checks: Detect conclusions that do not logically follow from the provided evidence or preceding arguments.
- Uncertainty and Ambiguity Handling: Distinguish demonstrably incorrect claims from unsupported, unverifiable, or ambiguous statements.
- Confidence-Calibrated Reporting: Communicate the strength of each finding without treating confidence as proof of correctness.
- Interactive Audit Dashboard: Display flagged claims, supporting evidence, error categories, and an overall assessment of the answer.

Google Developer Groups
On Campus • Fr. Conceicao Rodrigues College of Engineering

AROUND THE WORLD

BIT N BUILD



Blockchain

1. Incognito: Speak Without Being Seen.

Backstory:

Honest feedback about managers, teachers, and institutions is often suppressed because people fear retaliation. Traditional reporting systems rely on administrators to protect identities, while anonymous platforms struggle with fake submissions and abuse.
The challenge is to verify membership without revealing who submitted a report, even to the administrator.

Problem Statement:

Develop Incognito, a privacy-preserving reporting system that allows verified members to submit anonymous reports without revealing their identities.
The system must prevent outsiders and duplicate submissions while ensuring reports from the same member cannot be linked across different topics. It must also support abuse prevention without compromising anonymity.
Teams must demonstrate resistance to forged credentials, duplicate submissions, report correlation, and unauthorized identity disclosure.

Key Features:

- Anonymous Membership Proof: Verify that a reporter possesses a valid membership credential without disclosing their identity.
- Unlinkable Submissions: Prevent reports from being correlated across different topics or reporting categories.
- Privacy-Preserving Duplicate Control: Enforce per-member submission limits using topic-scoped cryptographic nullifiers or an equivalent mechanism.
- Administrator Privacy: Prevent administrators from directly recovering reporter identities from stored reports and verification records.
- Anonymous Abuse Prevention: Support content moderation, rate limiting, and credential revocation without routinely exposing reporter identities.
- Metadata Protection: Identify and mitigate identity leakage through application logs, timestamps, network metadata, and submission patterns.
- Security Evaluation: Test forged credentials, duplicate submissions, cross-topic correlation, unauthorized administrative access, and relevant collusion scenarios. Document the guarantees achieved, limitations, and remaining attack surfaces.

2. Fairwork: Get Paid for What You Deliver

Backstory:

Freelancers and clients frequently disagree over project milestones, incomplete deliverables, and payment releases. Freelancers risk not getting paid for completed work, while clients risk paying for work that does not meet agreed requirements. Traditional escrow services can introduce additional fees, delays, and dependence on centralized intermediaries.

Problem Statement:

Develop Fairwork, a blockchain-based agreement platform that connects freelance contracts with milestone-based payments. The system should enable clients and freelancers to establish verifiable agreements, track project progress, and release payments according to predefined conditions. It must also provide a transparent mechanism for handling disputes while protecting both parties from unfair outcomes.

Key Features:

- Milestone-Based Agreements: Define project deliverables, deadlines, acceptance criteria, and payment terms.
- Conditional Payments: Connect payments to predefined milestone conditions through smart contracts.
- Delivery Verification: Maintain verifiable records of submissions, approvals, and project progress.
- Dispute Resolution: Provide a structured process for resolving disagreements over deliverables and payments.
- Agreement Transparency: Maintain an auditable history of contract changes, decisions, and transactions.
- Participant Protection: Incorporate safeguards against missed deadlines, rejected deliverables, and unfair payment withholding.
- Payment Tracking: Provide visibility into pending, released, and disputed funds throughout a project's lifecycle.

3. Verity: Prove the Computation, Not Just the Result

Backstory:

Organizations increasingly rely on external services to perform computations and process sensitive data. However, verifying whether a service actually performed the agreed computation correctly can be difficult without independently repeating the entire process. Recording results alone does not guarantee their correctness, creating a need for verifiable and tamper-evident computation.

Problem Statement:

Develop Verity, a blockchain-based verifiable computation platform that enables one party to request a computation from another and independently verify that the returned result follows the agreed rules. The system should establish trust between parties without requiring them to blindly rely on the computation provider or repeat expensive computations themselves.

Key Features:

- Verifiable Computation: Establish mechanisms to prove that computational results follow the agreed execution rules.
- Independent Verification: Allow users to validate results without blindly trusting the provider.
- Tamper-Evident Records: Maintain verifiable records of computation requests, results, and verification outcomes.
- Input and Output Integrity: Ensure that the data and results associated with a computation can be independently authenticated.
- Dispute Resolution: Support challenges against suspicious results using verifiable evidence.
- Adversarial Testing: Demonstrate how the system detects manipulated outputs, invalid proofs, or inconsistencies between claimed and verified results.
- Execution Traceability: Provide a transparent history of computation requests, proof generation, and verification decisions.